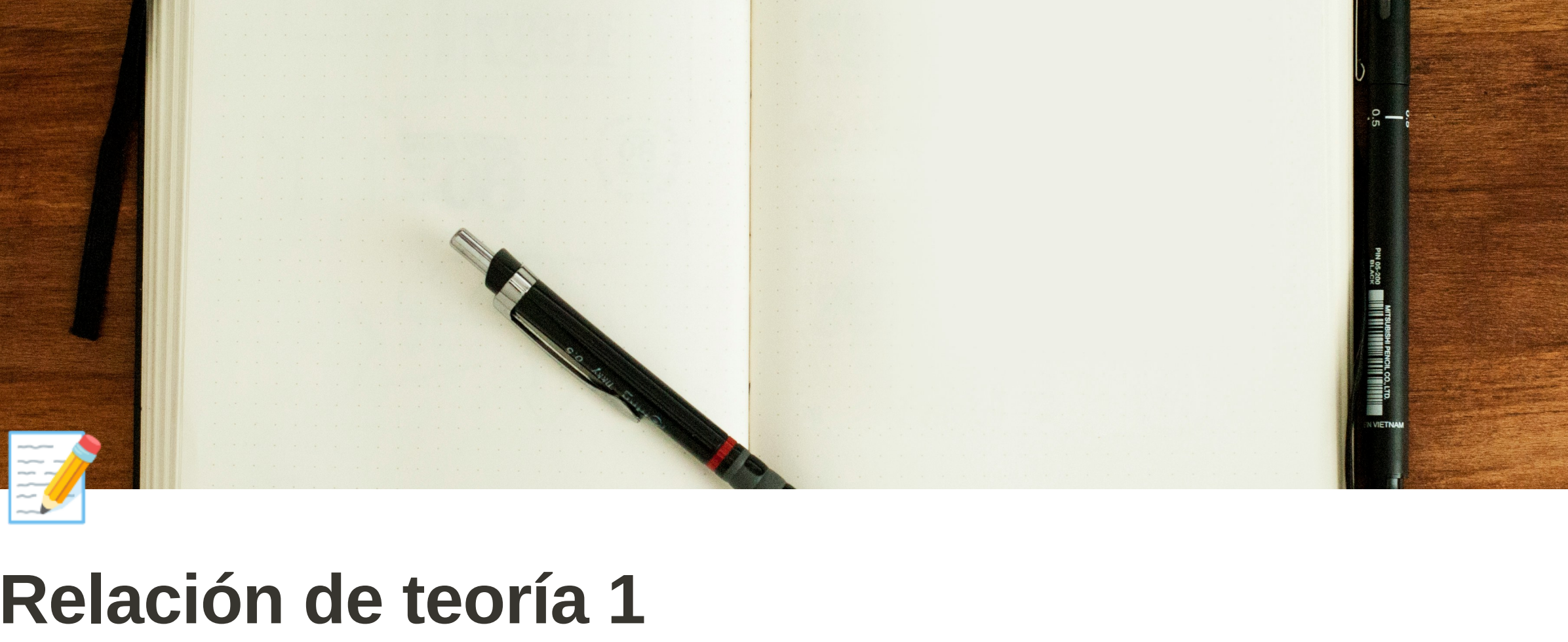




Relación de teoría 1

Ejercicio 14

14. Programa una sincronización correcta para el problema de la cena de los cinco filósofos, para lo cual simula el comportamiento de cada filósofo como un proceso concurrente diferente y sincroniza el acceso y la liberación de los recursos retenidos (palillos y silla) utilizando operaciones de semáforo.

```
Semaphore filosofo[0...5]; #5 excluido
Semaphore silla;

process proceso_filosofo(int index){
    while (true){
        silla.wait();
        filosofo[i].wait();
        filosofo[(i+1)%5].wait();

        #comer

        filosofo[i].signal();
        filosofo[(i+1)%5].wait();
        silla.signal();
    }
}

process main(){
    for (int i=0; i<5;i++){
        sem_init(filosofo,i);
    }
    sem_init(silla,4);
    .
    .
    .
}
```

Ejercicio 15

15. Se deben programar dos tipos de procesos (tipo A y tipo B), que simulan dos grupos distintos de personas que entran en una sala, con semáforos para implementar múltiples citas con diferentes números de participantes. Según la analogía de personas que entran y salen de una sala en equipos, una persona de tipo A no puede salir de la sala hasta que haya 10 personas de tipo B dentro y, de forma similar, una persona de tipo B no puede salir de la sala hasta que haya al menos una persona de tipo A y nueve personas de tipo B en la sala.

```
Semaphore tipo_A;
Semaphore tipo_B;
Semaphore exclusion;
sem_init(tipo_A,0);
sem_init(tipo_B,0);
sem_init(exclusion,1);

iniciada = false;
nA=0;
nB=0;

process proceso_tipo_A(int index){
    sem_wait(exclusion);
    nA++;

    while(nB<10){
        sem_signal(exclusion);
        sem_wait(tipo_A);
        sem_wait(exclusion);
    }

    for (int i=0; i<10;i++){
        sem_signal(tipo_B);
    }

    nA--;
    nB+=10;
    sem_signal(exclusion);
}

process proceso_tipo_B(int index){
    sem_wait(exclusion);
    nB++;

    while(nA<1 || nB<10){
        sem_signal(exclusion);
        sem_wait(tipo_B);
        sem_wait(exclusion);
    }

    sem_signal(tipo_A);
    sem_signal(exclusion);
}
```

60. Supongamos que estamos en una discoteca y resulta que está estropeado el servicio de chicas y todos tienen que compartir el de chicos. Se pretende establecer un protocolo de entrada al servicio usando semáforos que asegure siempre el cumplimiento de las siguientes restricciones:

- Chicas: sólo puede estar 1 dentro del servicio
- Chicos: pueden entrar más de 1, pero como máximo se admitirán a 5 dentro del servicio

- Versión machista del protocolo: los chicos tienen preferencia sobre las chicas. Esto quiere decir que si una chica está esperando entrar al servicio y llega un chico, este puede pasar y ella sigue esperando. Incluso si el chico que ha llegado no pudiera entrar inmediatamente porque ya hay 5 chicos dentro del servicio, sin embargo, pasará antes que la chica cuando salga algún chico del servicio.
- Versión feminista del protocolo: las chicas tienen preferencia sobre los chicos. Esto quiere decir que si un chico está esperando y llega una chica, ésta debe pasar antes. Incluso si la chica que ha llegado no puede entrar inmediatamente al servicio porque ya hay una chica dentro, pasará antes que el chico cuando salga la chica que está dentro.

Se pide: implementar las 2 versiones del protocolo anterior utilizando semáforos POSIX. Ayuda sobre la sintaxis de las operaciones de los semáforos (*no nombrados*) de POSIX 1003:

Versión machista

```
Semaphore cola_chicas = 0;
Semaphore cola_chicos = 0;
Semaphore exclusion = 1;

bool hay_chicas = false;
int n_chicos = 0;

process chicas(){
    sem_wait(exclusion);
    if (n_chicos!=0){
        sem_signal(exclusion);
        sem_wait(cola_chicas);
    }

    sem_wait(exclusion);
    hay_chicas = true;
    sem_signal(exclusion);

    #Uso el baño

    sem_wait(exclusion);
    hay_chicas = false;
    sem_signal(exclusion);

    if (n_chicos > 0) {
        sem_signal(cola_chicos);
    } else {
        sem_signal(cola_chicas);
    }
}

process chicos(){

    sem_wait(exclusion);

    if (hay_chicas){
        sem_signal(exclusion);
        sem_wait(cola_chicos);
    }

    n_chicos++;

    if (n_chicos < 5 && !hay_chicas){
        sem_signal(cola_chicos);
    }

    #Uso el baño

    sem_wait(exclusion);
    n_chicos--;

    if (n_chicos < 5){
        sem_signal(cola_chicos);
    } else {
        sem_signal(cola_chicas);
    }
    sem_signal(exclusion);
}
```

Version feminista

```
Semaphore cola_chicas = 0;
Semaphore cola_chicos = 0;
Semaphore exclusion = 1;

bool hay_chicas = false;
int n_chicos = 0;
int n_chicas = 0;

process chicas(){
    sem_wait(exclusion);
    if (n_chicos!=0 || hay_chicas){
        n_chicas++;
        sem_signal(exclusion);
        sem_wait(cola_chicas);
    }

    sem_wait(exclusion);
    hay_chicas = true;
    sem_signal(exclusion);

    #Uso el baño

    sem_wait(exclusion);
    hay_chicas = false;
    n_chicas--;
    sem_signal(exclusion);

    if (n_chicas > 0) {
        sem_signal(cola_chicas);
    } else {
        sem_signal(cola_chicos);
    }
}

process chicos(){

    sem_wait(exclusion);

    if (hay_chicas || n_chicas>0){
        sem_signal(exclusion);
        sem_wait(cola_chicos);
    }

    n_chicos++;

    if (n_chicos < 5 && (!hay_chicas || n_chicas >0)){
        sem_signal(cola_chicos);
    }

    #Uso el baño

    sem_wait(exclusion);
    n_chicos--;

    if (n_chicas > 0){
        sem_signal(cola_chicas);
    } else if (n_chicos<5){
        sem_signal(cola_chicos);
    }
    sem_signal(exclusion);
}
```

16. Utiliza semáforos para programar dos procesos que lean datos de un archivo. Por cada dato que lean, comprueban si el dato está en una tabla (leen directamente de la tabla) y, si es así, lo marcan (se escribe en la tabla); de lo contrario, terminan de leer y no hacen nada. Tenga en cuenta que un proceso finaliza cuando llega al final del archivo, que el acceso al archivo es por exclusión mutua y que dos procesos pueden leer en la tabla al mismo tiempo; pero si uno de los procesos está escribiendo, el otro no puede trabajar en la tabla, ni leer ni escribir. Tampoco puede darse el caso de que cuando un proceso está leyendo, el otro pueda empezar a escribir.

```
Semaphore exclusion = 1;
Semaphore escribir = 1;
Semaphore puede_leer = 1;
Tabla tabla;
bool escribiendo = false;

Process lector-escriptor(){
    while (!finArchivo){
        sem_wait(exclusion);
        dato = leer();
        sem_signal(exclusion);
        if (escribiendo){
            sem_wait(puede_leer);
        }
        if (find(dato,tabla)){
            sem_wait(escribir);
            escribiendo = true;
            sem_wait(puede_leer);
            escribir();
            escribiendo=false;
            sem_signal(puede_leer);
            sem_signal(escribir);
        }
    }
}
```

17. Demostrar que la siguiente sentencia tiene la poscondición $\{x \geq 0, x^2 = a^2\}$.
if $a > 0$ *then* $x := a$ *else* $x := -a$

Queremos probar la terna:

```
{true} if a>0 then x:=a else x:=-a {x>=0 ∧ x^2=a^2}
```

Mediante la regla de Hoare para el if tenemos que demostrar las siguientes ternas:

```
{a>0} x:=a {x>=0 ∧ x^2=a^2}
{a<=0} x:=-a {x>=0 ∧ x^2=a^2}
```

Demostramos primero la primera terna, mediante la regla de asignación

```
{a>=0 ∧ a^2=a^2} x:=a {x>=0 ∧ x^2=a^2}
```

Simplificamos

```
{a>=0} x:=a {x>=0 ∧ x^2=a^2}
```

Como $a > 0 \rightarrow a \geq 0$, por la regla de la consecuencia obtenemos

```
{a>0} x:=a {x>=0 ∧ x^2=a^2}
```

Ahora comprobamos la sentencia else:

```
{a<0} x:=-a {x>=0 ∧ x^2=a^2}
```

Aplicamos regla de asignación

```
{-a>=0 ∧ (-a)^2=a^2} x:=-a {x>=0 ∧ x^2=a^2}
```

```
{a<=0} x:=-a {x>=0 ∧ x^2=a^2}
```

Por la regla de la consecuencia obtenemos:

```
{a<=0} x:=-a {x>=0 ∧ x^2=a^2}
```

Así, como tenemos las ternas:

```
{a>0} x:=a {x>=0 ∧ x^2=a^2}
{a<=0} x:=-a {x>=0 ∧ x^2=a^2}
```

A través de la regla del if podemos concluir que:

```
{true} if a>0 then x:=a else x:=-a {x>=0 ∧ x^2=a^2}
```